VIT®
BHOPAL
www.vitbhopal.ac.in

Software Engineering (CSE3005)

Prepared by: Dr. Rajneesh Kumar Patel

Module-1

Module 1: Software Process Models

- **The Nature of Software**
  - ✔ A generic view of process
  - ✔ A layered Technology
  - ✔ A Process Framework– Product and Process
- **Software Process Models**
  - ✔ Waterfall Model
  - ✔ Incremental Process Models
  - ✔ Evolutionary Process Models
  - ✔ Prototyping-Spiral Model
  - ✔ The RAD Model
  - ✔ Concurrent Model
  - ✔ The Concurrent Development Model
  - ✔ Specialized Process Models
  - ✔ The Unified Process

- **Introduction to Agile Process**
  - ✔ Agile Management Practices
  - ✔ Risk Management and the Customer in Agile Methods
  - ✔ Agile Engineering Practices
  - ✔ Tailoring and Improving Agile Methods
  - ✔ Miscellaneous Agile Methods
  - ✔ Challenges in Adopting Agile Methods

Software Crisis

It was in late 1960's many software projects failed.

- Many software projects late, over budget, providing unreliable software that is expensive to maintain.
- Many software projects produced software which did not satisfy the requirements of the customer.
- Complexities of software projects increased as hardware capability increased. Larger software system is more difficult and expensive to maintain.
- Demand of new software increased faster than ability to generate new software.

All the above attributes of what was called a **'Software Crisis'.** So the term 'Software Engineering' first introduced at a conference in late 1960's to discuss the software crisis.

Why software engineering?

Once the need for software engineering was identified and software engineering recognized as a discipline.

- The late 1970's saw the widespread evolution of software engineering principles.
- The 1980's saw the automation of software engineering and growth of CASE (Computer Aided Software Engineering).
- The 1990's have seen increased emphasis on the 'management' aspects of projects and the use of standard quality and 'process' models like ISO 9001 and the Software Engineering Institute's Software Capability Maturity Model (CMM).
- These models help organizations put their software development and management processes in place.

Software and Software Engineering

The term software engineering is composed of two words, **software and engineering**.

Software: is more than just a program code. Software is considered to be a collection of executable programming code, associated libraries and documentations, which serves some computational purpose.

it is defined as-

1. **Instructions :** Programs that when executed provide desired function, features, and performance

2. **Data structures :** Enable the programs to adequately manipulate information

3. **Documents:** Descriptive information in both hard copy and virtual forms that describes the operation and use of the programs.

Software and Software Engineering

Engineering: It is all about developing products, using well-defined, scientific principles and methods.

- Engineering is the process of designing and building something that serves a particular purpose and finds a **cost-effective solution to problems**.
- **Engineering** is the application of **scientific** and **practical** knowledge to **invent, design, build, maintain**, and **improve frameworks, processes, etc**.
- Application of science tools and methods to find **cost effective solution to the problem**.

Software Engineering

- **Software Engineering** is an engineering branch related to the evolution of software product using well-defined scientific principles, techniques, and procedures. **The result of software engineering is an effective and reliable software product.**
- It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software.
- It is defined as a **systematic disciplined** and **quantifiable** approach to the development, operations and maintenance of software.

IEEE defines software engineering as:

- The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software.

Need of Software Engineering required

The need of software engineering arises because of higher rate of change in user requirements and environment on which the software is working.

Software Engineering is required due to the following reasons:

- To manage Large software
- For more Scalability
- Cost Management
- To manage the dynamic nature of software
- For better quality Management

Software Paradigms

Need of Software Engineering required

- **Large software -** It is easier to build a wall than to a house or building, likewise, as the size of software become large engineering has to step to give it a scientific process.
- **Scalability-** If the software process were not based on scientific and engineering concepts, it would be easier to re-create new software than to scale an existing one.
- **Cost-** As hardware industry has lower down the price of computer and electronic hardware. But the cost of software remains high if proper process is not adapted.
- **Dynamic Nature-** The always growing and adapting nature of software hugely depends upon the environment in which the user works. Thus, new enhancements need to be done in the existing one. This is where software engineering plays a good role.
- **Quality Management-** Better process of software development provides better and quality software product.

Characteristics of Software

A software product can be judged by what it offers and how well it can be used. This software must satisfy on the following grounds:

- **Operational:** This tells us how well software works in operations. It can be measured on
- **Transitional:** This aspect is important when the software is moved from one platform to another
- **Maintenance:** This aspect briefs about how well a software has the capabilities to maintain itself in the ever changing environment:

Characteristics of Software

Operational

- Budget
- Usability
- Efficiency
- Correctness
- Functionality
- Dependability
- Security
- Safety

Transitional

- Portability
- Interoperability
- Reusability
- Adaptability

Maintenance

- Modularity
- Maintainability
- Flexibility
- Scalability

Characteristics of Software

Three characteristics of software that makes it different from hardware:

- Software is **developed** or **Engineered**, it is not **manufactured** in the classical sense.
- Software doesn't "**wear out**".
- Although the industry is moving toward **component-based assembly**, most software continues to be **custom built**.

Characteristics of Software

1. Software is developed or Engineered, it is not manufactured in the classical sense.

- Although some similarities exist between software development and hardware manufacturing, but few activities are fundamentally different.
- In both activities, high quality is achieved through good design, but the manufacturing phase for hardware can introduce quality problems than software.

Characteristics of Software

2. Software doesn’t “wear out”.

- Hardware components suffer from the environmental effects like dust, vibration, abuse, temperature extremes, etc. But, Softwares are not depend to the environmental maladies that cause hardware to wear out.
- When a hardware component wears out, it is replaced by a spare part. There are no software spare parts.
- Every software failure indicates an error in design or in the process. Therefore, the software maintenance tasks involve considerably more complexity than hardware maintenance.

Characteristics of Software

3. Although the industry is moving toward component-based assembly, most software continues to be custom built.

- A software component should be designed and implemented so that it can be reused in many different programs.
- Modern reusable components encapsulate both data and the processing that is applied to the data, enabling the software engineer to create new application form reusable parts.

“In short, Software engineering is a branch of computer science, which uses well-defined engineering concepts required to produce efficient, durable, scalable, in-budget and on-time software products.”

Software Engineering - A Layered Technology

In order to build software that is ready to meet the challenges of the twenty-first century, you must recognize a few simple realities:

- Problem should be understood before software solution is developed
- Design is an important Software Engineering activity
- Software should exhibit high quality
- Software should be maintainable

These simple realities lead to one conclusion. Software in all of its forms and across all of its application domains should be **engineered**.

Software Engineering - A Layered Technology

IEEE has developed a more comprehensive definition as :

1) Software engineering is the application of a **systematic**, **disciplined**, **quantifiable** approach to the **development**, **operation**, and **maintenance** of software.

2) The study approaches as in (1), Software Engineering is a **layered technology.** Software Engineering encompasses a **Process, Methods** for **managing and engineering software and tools.**

Software Engineering - A Layered Technology

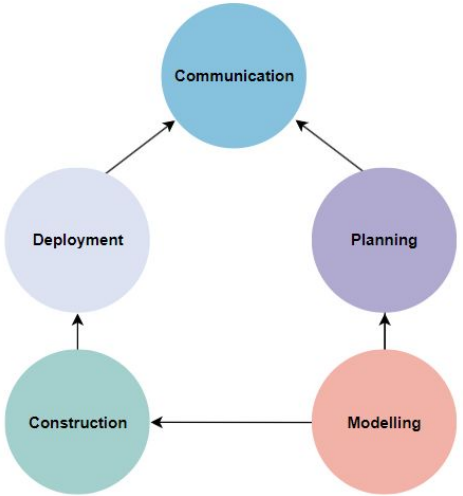
The following Figure represents **Software engineering Layers**

Software Engineering - A Layered Technology

- Referring to the Figure, any **engineering approach** must rest on an organizational commitment to **quality.** The **bedrock** that supports software engineering is a **quality focus**.
- The **foundation** for software engineering is the **process layer**. **Process** defines a **framework** that must be established for **effective delivery of software engineering technology.**
- Software engineering **methods** provide the technical **how-to's for building software.** Methods encompass a **broad array of tasks** that include communication, requirements analysis, design modelling, program construction, testing, and support.
- Software engineering **tools** provide **automated or semi automated support** for the process and the methods.

Generic Process Framework

- The **process framework** encompasses a **set of umbrella activities** that are applicable across the **entire software process**.
  - A **generic process framework** for software engineering encompasses **five activities:**
    - Communication
    - Planning
    - Modelling
    - Construction
    - Deployment



Generic Process Framework

1. Communication

- In this step, we communicate with the clients and end-users.
- We discuss the requirements of the project with the users.
- The users give suggestions on the project. If any changes are difficult to implement, we work on alternative ideas.

2. Planning

- In this step, we plan the steps for project development. After completing the final discussion, we report on the project.
- Planning plays a key role in the software development process.
- We discuss the risks involved in the project.

3. Modelling

- In this step, we create a model to understand the project in the real world. We showcase the model to all the developers. If changes are required, we implement them in this step.
- We develop a practical model to get a better understanding of the project.

Generic Process Framework

4. Construction

- In this step, we follow a procedure to develop the final product.
- If any code is required for the project development, we implement it in this phase.
- We also test the project in this phase.

5. Deployment

- In this phase, we submit the project to the clients for their feedback and add any missing requirements.
- We get the client feedback.
- Depending on the feedback form, we make the appropriate changes.

Software Engineering Process Framework

Umbrella Activities are applied throughout a software project and **help** a software team to manage and control progress, quality, change, and risk. Typical umbrella activities include:

- Software project tracking and control
- Risk management
- Software quality assurance
- Technical reviews
- Measurement
- Software configuration management
- Reusability management
- Work product preparation and production

The Software Engineering Practice

The Essence of Practice:

- **Understand the problem** (communication and analysis)
- **Plan a solution** (software design)
- **Carry out the plan** (code generation)
- **Examine the result for accuracy** (testing and quality assurance)

The Software Engineering Practice

Understand the problem (communication and analysis)

- Who are the stakeholders?
- What functions and features are required to solve the problem?
- Is it possible to create smaller problems that are easier to understand?
- Can a graphic analysis model be created?

Plan the Solution (software design)

- Have you seen similar problems before?
- Has a similar problem been solved?
- Can readily solvable sub problems be defined?
- Can a design model be created?

The Software Engineering Practice

Carry Out the Plan (code generation)

- Does solution conform to the plan?
- Is each solution component provably correct?

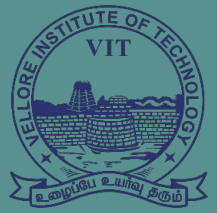
Examine the Result (testing and quality assurance)

- Is it possible to test each component part of the solution?
- Does the solution produce results that conform to the data, functions, and features required?

Software General Principles

David Hooker has Proposed seven principles that focus on software Engineering practice.

- The First Principle: **The Reason It All Exists**
- The Second Principle: **Keep It Simple**
- The Third Principle: **Maintain the Vision**
- The Fourth Principle: **What You Produce, Others Will Consume**
- The Fifth Principle: **Be Open to the Future**
- The Sixth Principle: **Plan Ahead for Reuse**
- The Seventh principle: **Think!**



VIT®
BHOPAL
www.vitbhopal.ac.in

Software Process Models

Prepared by: Dr. Rajneesh Kumar Patel

Process Models

- **Software process** can be defined as the **structured set of activities** that are required to **develop the software system**.

- To **solve actual problems** in an industry, a software engineer or a **team of engineers** must incorporate a **development strategy** that encompasses the **process, methods, and tools layers.** This strategy is often **referred to as a process model** or a software engineering paradigm.

- A **process model** for software engineering is **chosen** based on the **nature of the project** and application, the methods and tools to be used, and the controls and deliverables that are required.

Goal of Software Process Models

The goal of a software process model is to **provide guidance** for **systematically coordinating** and **controlling** the **tasks** that must be performed in order to **achieve** the **end product** and their project objectives.

A process model defines the following:

- A set of tasks that need to be performed.
- The inputs and output from each task.
- The preconditions and postconditions for each task.
- The sequence and flow of these tasks.

Software Product

- A software product, **user interface** must be **carefully designed and implemented** because **developers** of that product and **users** of that product are **totally different**.
- In case of a **program**, very **little documentation** is expected, but a **software product** must be **well documented**.
- Various Operational Characteristics of software are:
    - Correctness
    - Usability/Learnability
    - Integrity
    - Reliability
    - Efficiency
    - Security

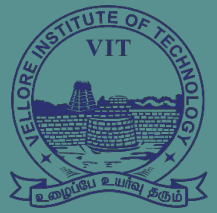
Difference between software process and software product

Software Process

- Processes are developed by individual user and it is used for personal use.
- Process may be small in size and possessing limited functionality.
- Process is generally developed by process engineers.
- Only one person uses the process, hence lack of user interface

Software Product

- It is developed by multiple users and it is used by large number of people or customers.
- It consists of multiple program codes; relate documents such as SRS, designing documents, user manuals, test cases.
- Process is generally developed by process engineers. Therefore systematic approach of developing software product must be applied.
- Multiuser no lack of user interface.



VIT®
BHOPAL
www.vitbhopal.ac.in

SDLC & Waterfall Model

Prepared by: Dr. Rajneesh Kumar Patel

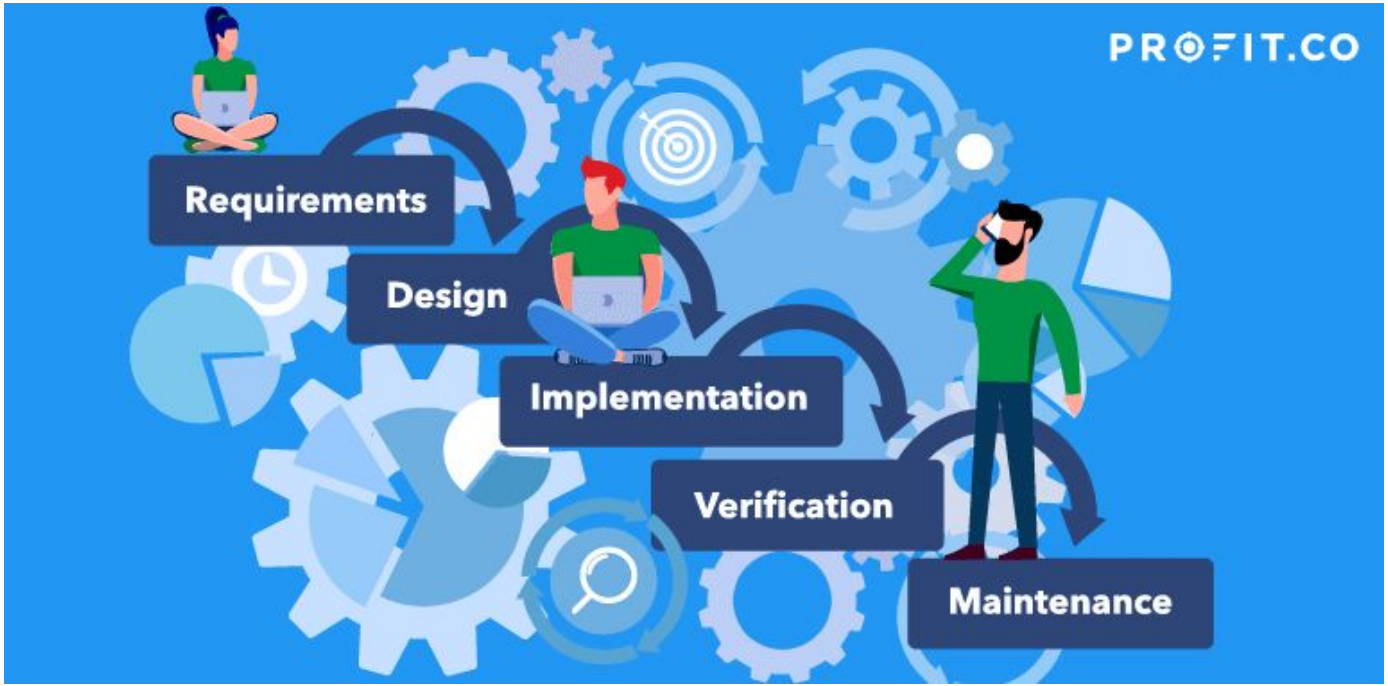
Software Development Life Cycle (SDLC)

- **SDLC** is a **well-defined**, **structured sequence of stages** in software engineering to develop the intended **software product**.
- Software Development Life Cycle (SDLC) is a process used by software industry to design, develop and test high quality software.
- The SDLC aims to produce high-quality software that meets or exceeds customer expectations, reaches completion within times and cost estimates.
- Each **phase** has **various activities** to develop the software product. It also **specifies the order** in which each **phase must be executed**.
- A software life cycle model is **either** a **descriptive or prescriptive** characterization of how software is or should be developed. A descriptive model describes the history of how a particular software system was developed.

Waterfall model or linear sequential model or classic life cycle model

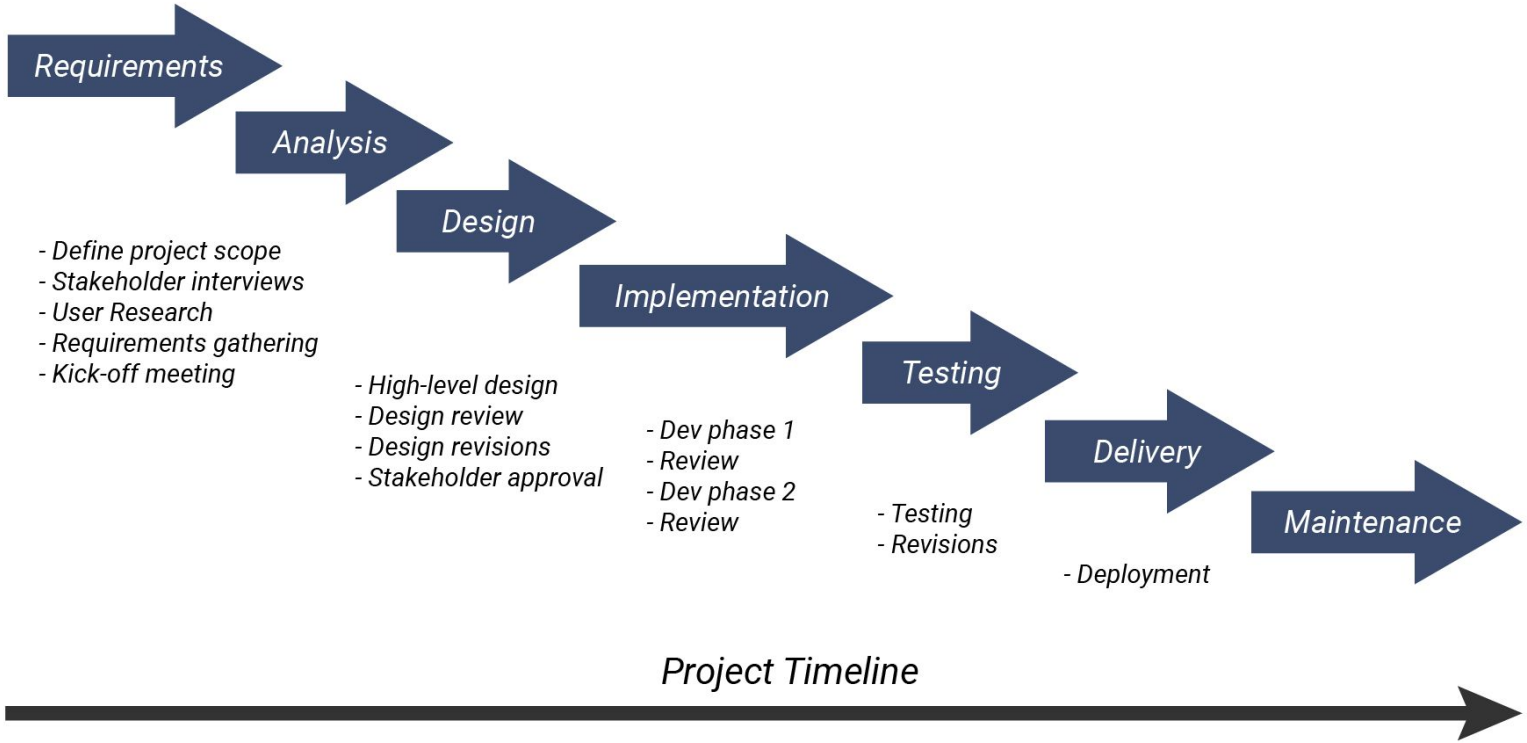
- The **Waterfall approach** was established in **1970** by **Winston w. Royce.**
- It contains **five phases** of management, where each **requires a deliverable from the previous phase to proceed.**
- Waterfall is **ideal for projects** like software development, where the **end result is clearly established before starting.**
- The **linear sequential model** suggests a **systematic, sequential approach** to software development that **begins** at the **system level** and **progresses through analysis, design, coding, testing, and maintenance.**

Waterfall model or linear sequential model or classic life cycle model



Waterfall

(Plan Driven)



Waterfall Model

- **Requirements and Planning**
  - The requirements and planning phase of waterfall project management identifies what the project should do.
  - The project manager tries to understand the project's requirements based on what the project sponsors need.
  - This phase involves identifying and describing the project's risks, assumptions, dependencies, quality metrics, costs, and timeline.

- **Design**
  - The design phase designs and documents all your decisions. In this case, you develop solutions that can solve the project's requirements.
  - The best way to do so is to note all the actions you'll take to deliver the project scope to execute them.
  - Design covers the project's schedule, budget, and objectives, and you can think of design as a blueprint or road map to the complete project.

Waterfall Model

- **Implementation**
  - The implementation phase executes your project plan and design to produce the desired product.
  - If your company develops software, you will spend this phase coding the software functionalities. Or, if you're managing a project at a construction company, you will construct a house in this phase.
  - Implementation takes up a significant portion of waterfall project management. Everything that happens during this phase should be carefully documented.

- **Verification/Testing**
  - Testing verifies that the product developed in the implementation phase fulfils the entire project's requirements.
  - If this is not the case, the project team must review the project from phase one to identify what went wrong.
  - The testing phase uses various quality metrics and customer satisfaction to measure the project's success.

Waterfall Model

- **Maintenance**
  - The maintenance phase extends beyond the five stages of project management into the project's lifetime.
  - This phase involves making minor modifications to improve the product developed during implementation and performing other routine maintenance tasks.
  - It's also a phase to identify any errors you might have missed during the testing phase.

Waterfall Model

Advantages of waterfall model:

- This model is simple and easy to understand and use.
- Waterfall model works well for **smaller projects** where **requirements are very well understood.**
- Each phase proceeds sequentially.
- **Documentation** is produced at **every stage** of the software's development. This makes understanding the product designing procedure, simpler.
- After every major stage of software coding, testing is done to check the correct running of the code. help us to control schedules and budgets.

Waterfall Model

Disadvantages of waterfall model:

- Not a good model for **complex, object-oriented projects** and long projects.
- **Not suitable** for the projects where **requirements** are at a moderate to **high risk of changing**.
- High amounts of **risk and uncertainty**.
- Customer can see **working model** of the project only **at the end**. after reviewing of the working model **if the customer gets dissatisfied then it causes serious problem**.
- **You cannot go back a step** if the design phase has gone wrong, things can get very complicated in the implementation phase.

VIT®
BHOPAL
www.vitbhopal.ac.in

INCREMENTAL MODEL

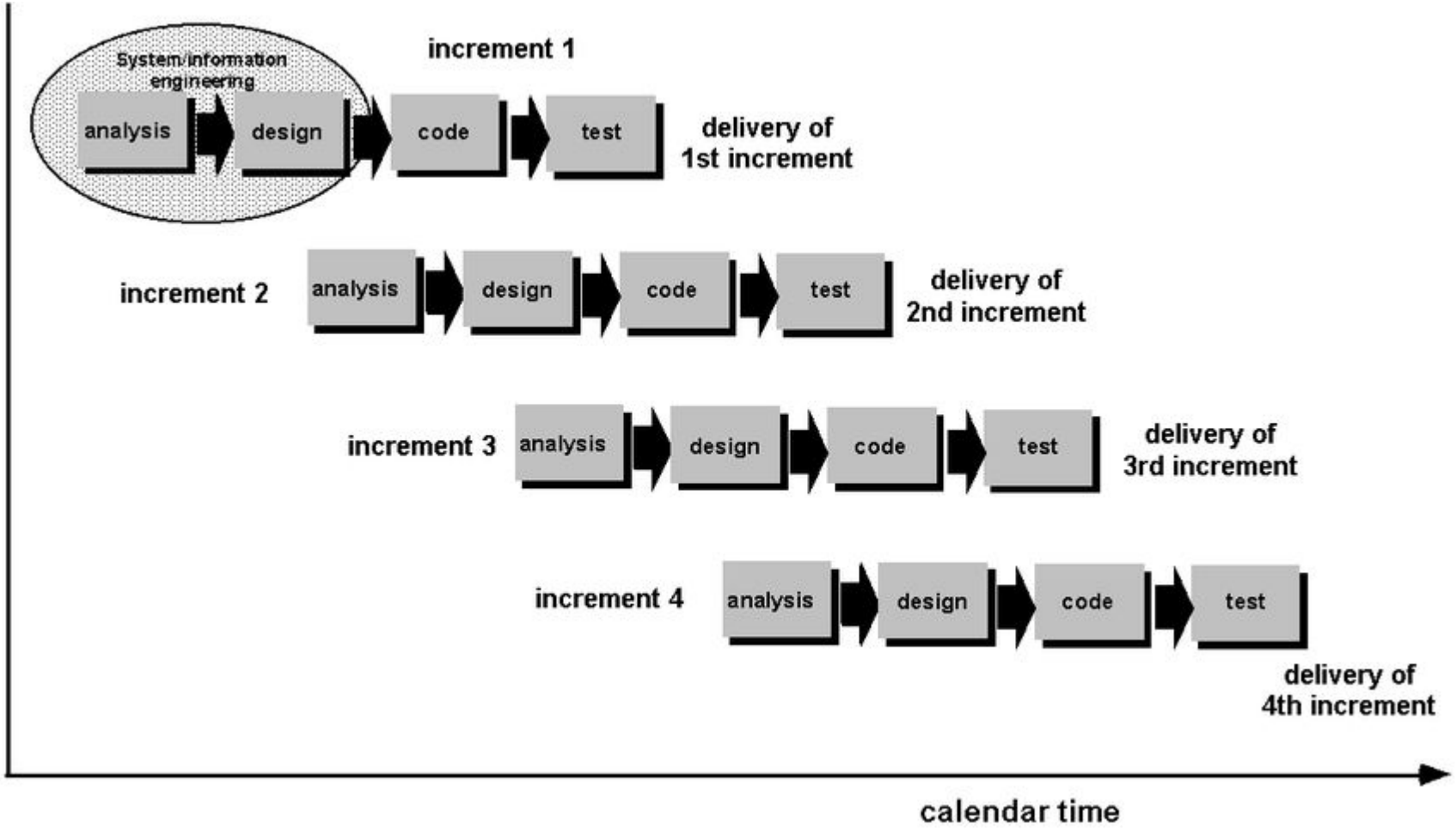
Prepared by: Dr. Rajneesh Kumar Patel

INCREMENTAL MODEL

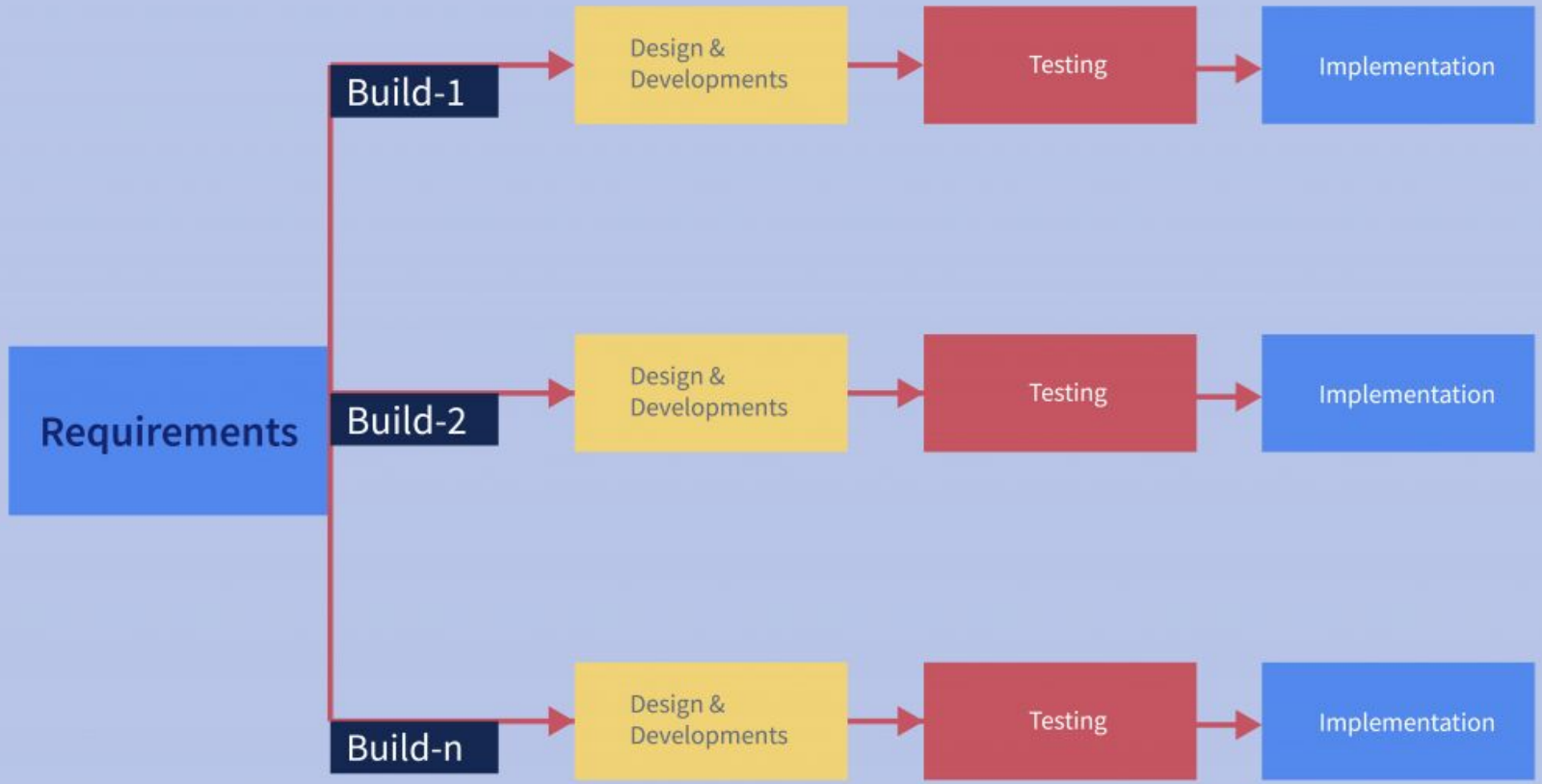
- The incremental model combines the **elements of waterfall model** and they are applied in an **iterative fashion.**
- The **first increment** in this model is generally a **core product.**
- **Each increment builds** the product and submits it to the **customer** for any suggested **modifications**.
- The **next increment** implements on the **customer's suggestions** and **add additional requirements** in the previous increment.
- This process is repeated until the product is finished.

INCREMENTAL MODEL

- **When to use the Incremental model:**
    - This model can be used when the requirements of the complete system are clearly defined and understood.
    - Major requirements must be defined; however, some details can evolve with time.
    - There is a need to get a product to the market early.
    - There are some high-risk features and goals.


System/information engineering
increment 1
analysis
design
code
test
delivery of 1st increment
increment 2
analysis
design
code
test
delivery of 2nd increment
increment 3
analysis
design
code
test
delivery of 3rd increment
increment 4
analysis
design
code
test
delivery of 4th increment
calendar time

Incremental Model



InterviewBit

INCREMENTAL MODEL

- **Advantages of Incremental model: -**
  - Generates working software quickly and early during the software life cycle.
  - This model is more flexible – less costly to change scope and requirements.
  - It is easier to test and debug during a smaller iteration.
  - In this model customer can respond to each built.
  - There is low risk for overall project failure.

INCREMENTAL MODEL

Disadvantages of Incremental model: -

- Needs good planning and design at the management a technical level.
- Needs a clear and complete definition of the whole system before it can be broken down and built incrementally.
- Total cost is higher than waterfall.
- Time foundation create problem to complete the project.

VIT®
BHOPAL
www.vitbhopal.ac.in

Iterative Waterfall Model

Prepared by: Dr. Rajneesh Kumar Patel

Iterative Waterfall Model

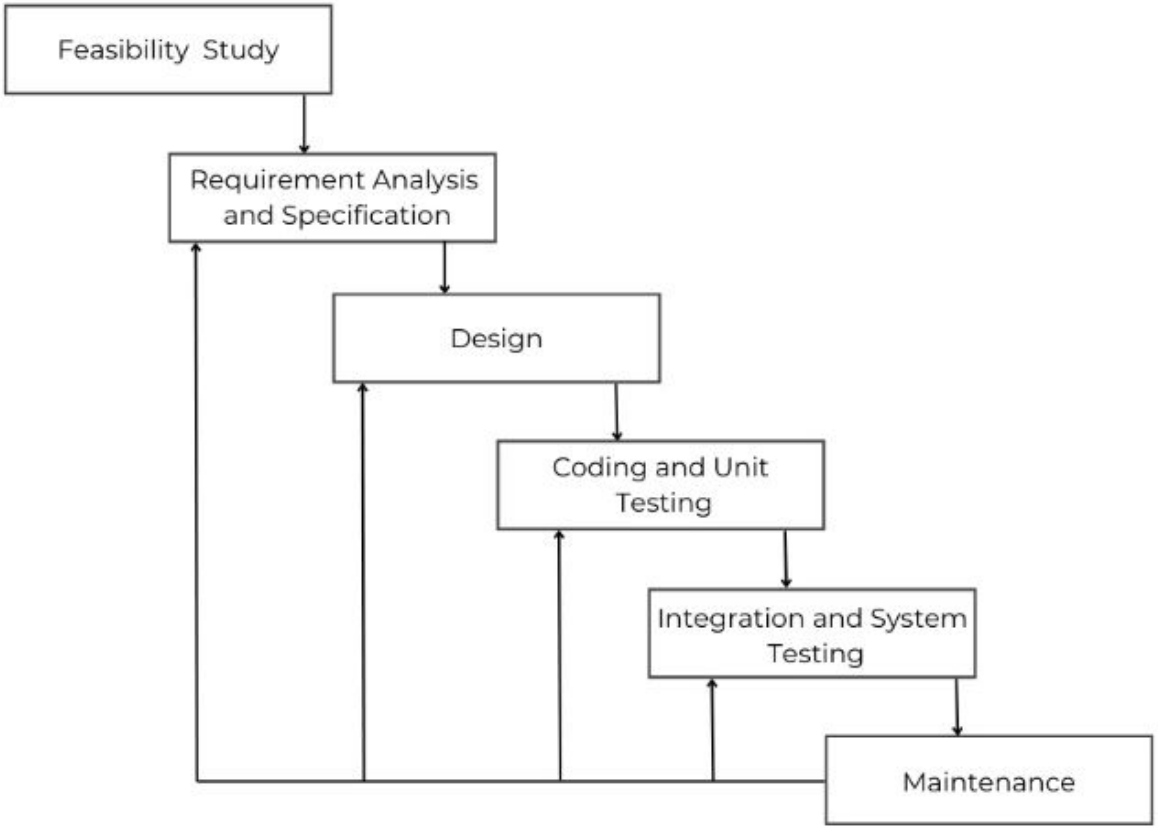
- In a practical software development project, the classical waterfall model is hard to use. So, the Iterative waterfall model can be thought of as incorporating the necessary changes to the classical waterfall model.

- It is almost the same as the classical waterfall model except some changes are made to increase the efficiency of the software development.

- **The iterative waterfall model provides feedback paths from every phase to its preceding phases, which is the main difference from the classical waterfall model.**

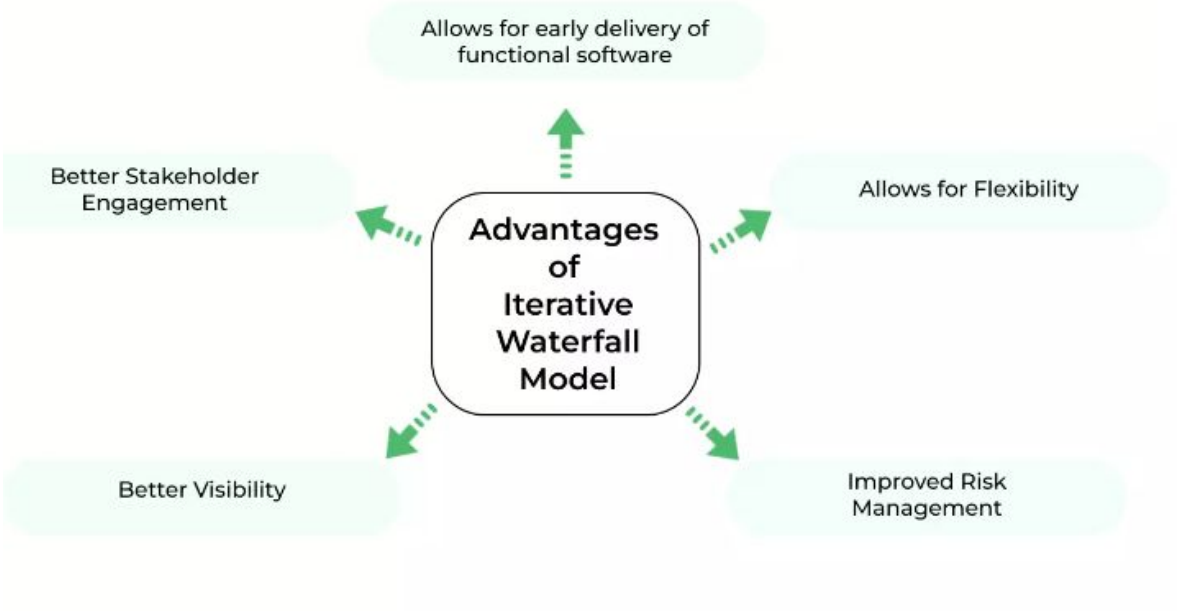
- Feedback paths introduced by the iterative waterfall model are shown in the figure.

ITERATIVE WATERFALL MODEL



Advantages of Iterative Waterfall Model

It is easy to understand and implement, making it a good choice for small projects or teams that are new to software development.



Disadvantages of Iterative Waterfall Model

1- Inflexibility
2- Lack of Customer involvement
3- Risk of Failure
4- Poor Communication
5- High Costs

VIT®
BHOPAL
www.vitbhopal.ac.in

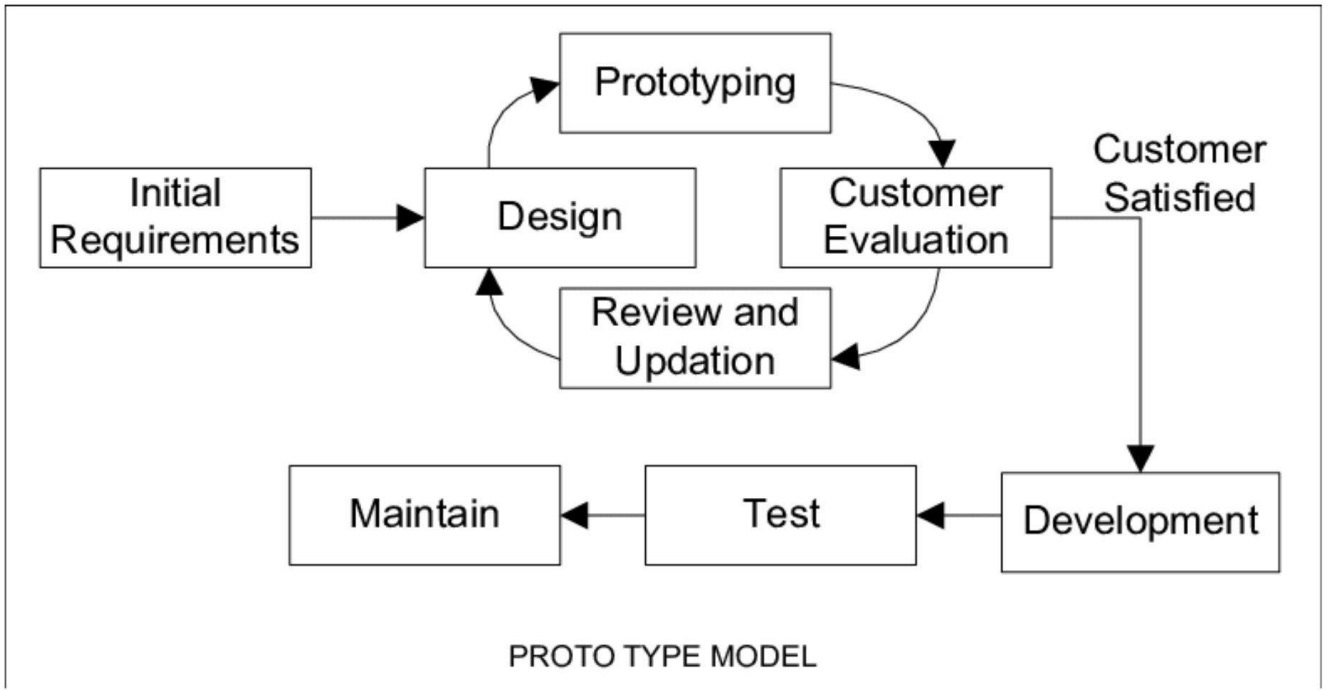
Prototyping MODEL

Prepared by: Dr. Rajneesh Kumar Patel

Prototyping Model

- Prototyping is defined as the process of developing a working replication of a product or system that has to be engineered.
- It offers a small scale facsimile of the end product and is used for obtaining customer feedback.
- This model is used when the customers do not know the exact project requirements beforehand.
- A prototype of the end product is first developed, tested and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.

Prototyping Model



Need of Prototyping Model

- The Prototyping Model should be used when the requirements of the product are not clearly understood or are unstable.
- The prototyping model can also be used if requirements are changing quickly.
- This is a valuable mechanism for gaining better understanding of the customers needs:
  - How the screens might look like
  - How the user interface would behave
  - How the system would produce outputs
- A prototyping model can be used when technical solutions are unclear to the development team.

Need of Prototyping Model

- A developed prototype can help engineers to critically examine the technical issues associated with the product development.
- Often, major design decisions depend on issues like the response time of a hardware controller, or the efficiency of a sorting algorithm, etc.
- In such circumstances, a prototype may be the best or the only way to resolve the technical issues.
- A prototype of the actual product is preferred in situations such as:
  - User requirements are not complete.
  - Technical issues are not clear.

Advantages of Prototyping Model

- The customers get to see the partial product early in the life cycle. This ensures a greater level of customer satisfaction and comfort.
- New requirements and missing functionalities can be easily accommodated as there is scope for refinement.
- Errors can be detected much earlier thereby saving a lot of effort and cost, besides enhancing the quality of the software.
- Prototyping can help bridge the gap between technical and non-technical stakeholders, improving overall project efficiency and effectiveness.
- The developed prototype can be reused by the developer for more complicated projects in the future.
- Prototyping can help reduce the risk of project failure by identifying potential issues and addressing them early in the process.

Disadvantages of Prototyping Model

- Costly with respect to time as well as money.
- There may be too much variation in requirements each time the prototype is evaluated by the customer. Hence, It is very difficult for developers to accommodate all the changes demanded by the customer.
- Poor Documentation due to continuously changing customer requirements.
- The prototype may not reflect the actual business requirements of the customer, leading to dissatisfaction with the final product.
- The prototype may not consider technical feasibility and scalability issues that can arise during the final product development.
- The prototype may be developed using different tools and technologies, leading to additional training and maintenance costs.

VIT®
BHOPAL
www.vitbhopal.ac.in

SPIRAL MODEL

Prepared by: Dr. Rajneesh Kumar Patel

SPIRAL MODEL

- **Spiral model** is one of the most important Software Development Life Cycle models, which provides support for **Risk Handling**.
- it looks like a spiral with many loops.
- The exact number of loops of the spiral is unknown and can vary from project to project.
- Each loop of the spiral is called a **Phase of the software development process.**
- The Spiral Model is a risk-driven model, meaning that the focus is on managing risk through multiple iterations of the software development process.

SPIRAL MODEL

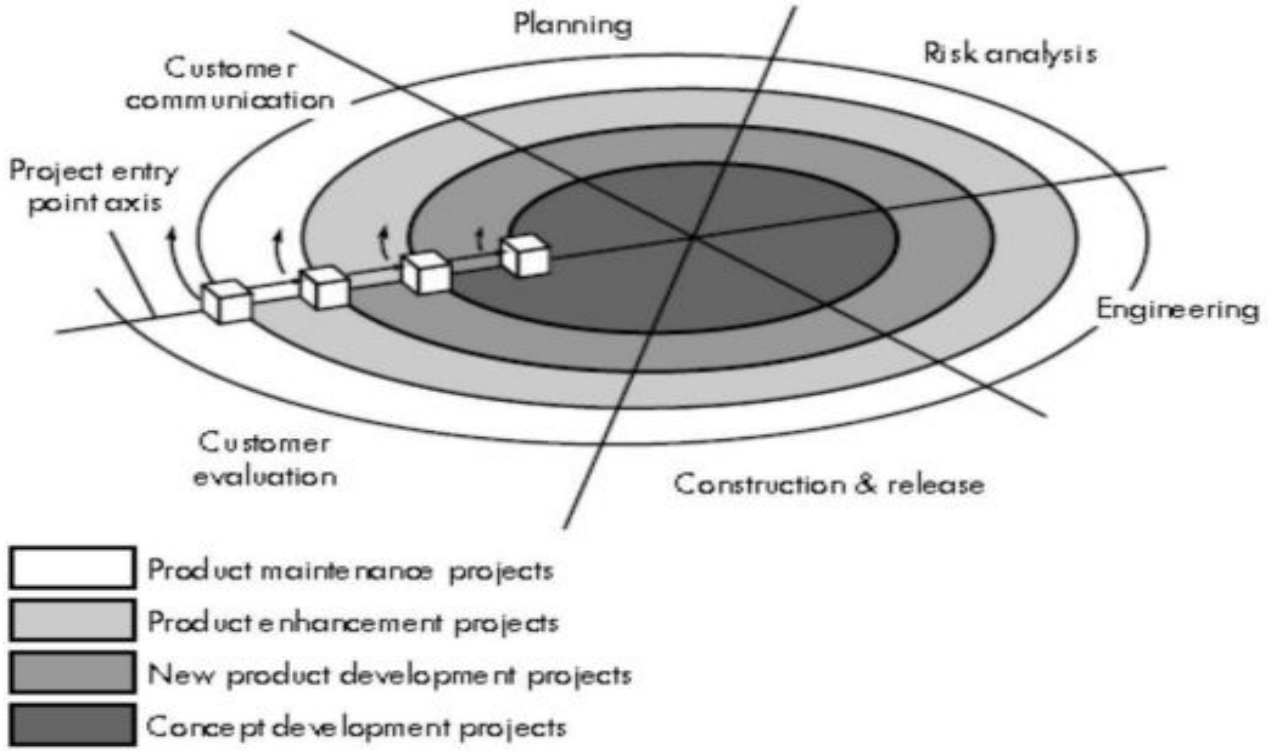
- **When to use Spiral model:**
  - When costs and risk evaluation is important.
  - For medium to high-risk projects.
  - Long-term project commitment unwise because of potential changes to economic priorities.
  - Users are unsure of their needs.
  - Requirements are complex.
  - New product line.
  - Significant changes are expected (research and exploration).


Plan
Requirements Gathering
Risk Analysis
Risk Reduction Prototyping
Start
01
02
03
04
Release
Evaluate
Customer Evolution
Engineering
Coding Testing

SPIRAL MODEL

- **Spiral Model contains six task regions:**
  - **Customer communication**—tasks required to establish effective communication between developer and customer.
  - **Planning**—tasks required to define resources, time lines, and other project related information.
  - **Risk analysis**—tasks required to assess both technical and management risks.
  - **Engineering**—tasks required to build one or more representations of the application.
  - **Construction and release**—tasks required to construct, test, install, and provide user support(e.g., documentation and training).
  - **Customer evaluation**—tasks required to obtain customer feedback based on evaluation of the software representations created during the engineering stage and implemented during the installation stage

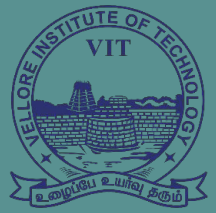
SPIRAL MODEL



SPIRAL MODEL

- **Advantages of Spiral model: -**
  - High amount of risk analysis hence, avoidance of Risk is enhanced.
  - Good for large and mission-critical projects.
  - Strong approval and documentation control.
  - Additional Functionality can be added at a later date.
  - Software is produced early in the software life cycle.

- **Disadvantages of Spiral model: -**
  - Can be a costly model to use.
  - Risk analysis requires highly specific expertise.
  - Project’s success is highly dependent on the risk analysis phase.
  - Doesn’t work well for smaller projects.



VIT®
BHOPAL
www.vitbhopal.ac.in

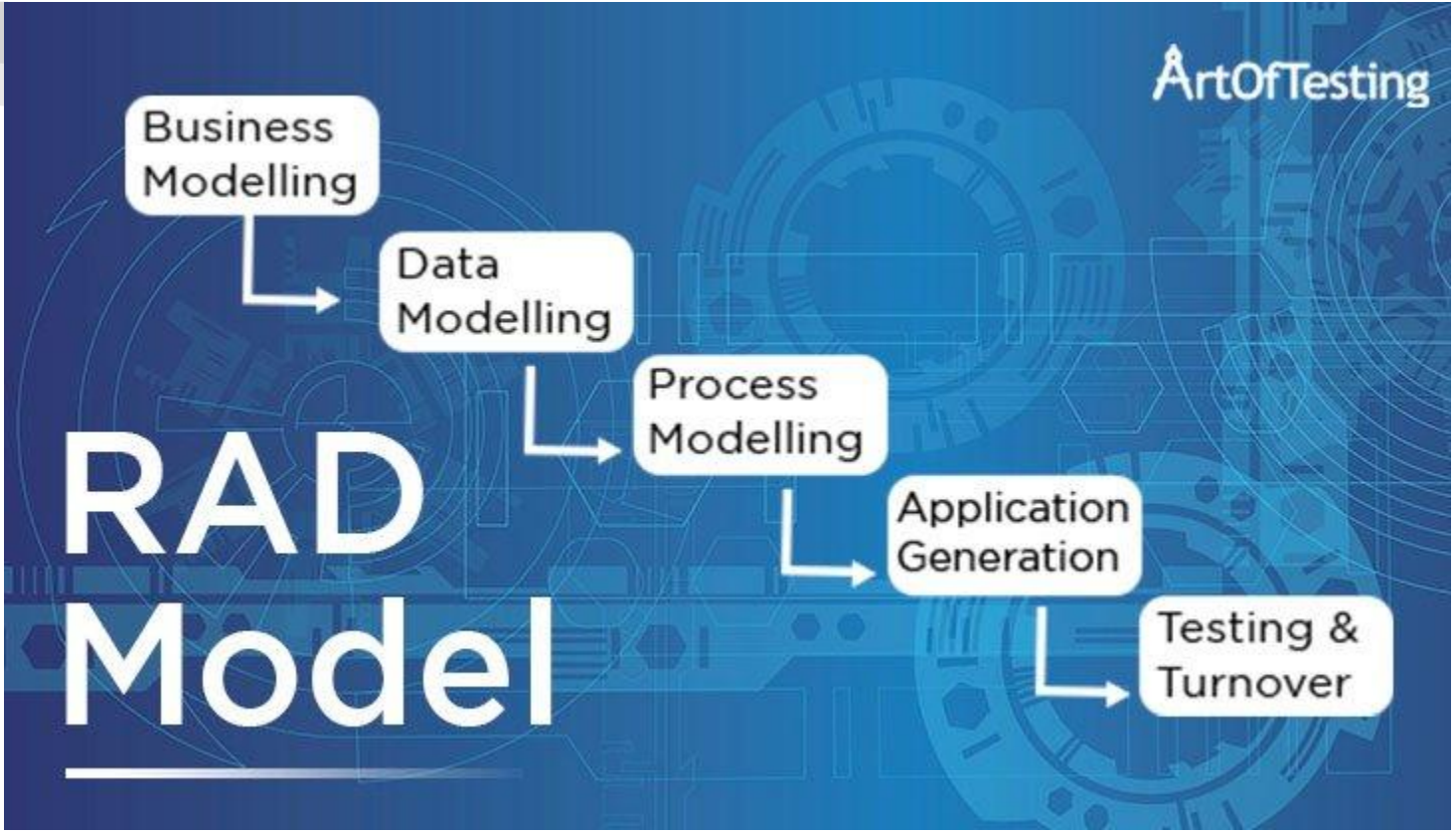
RAD (Rapid Application Development) Model

Prepared by: Dr. Rajneesh Kumar Patel

RAD (Rapid Application Development) Model

- RAD is a linear sequential software development process model.
- The RAD model is based on prototyping and iterative development with no specific planning involved.
- RAD is a software development methodology that uses minimal planning in favor of rapid prototyping.
- The functional modules are developed in parallel as prototypes and are integrated to make the complete product for faster product delivery.
- The most important aspect for this model to be successful is to make sure that the prototypes developed are reusable.
- **RAD model distributes the analysis, design, build and test phases into a series of short, iterative development cycles.**

RAD (Rapid Application Development) Model

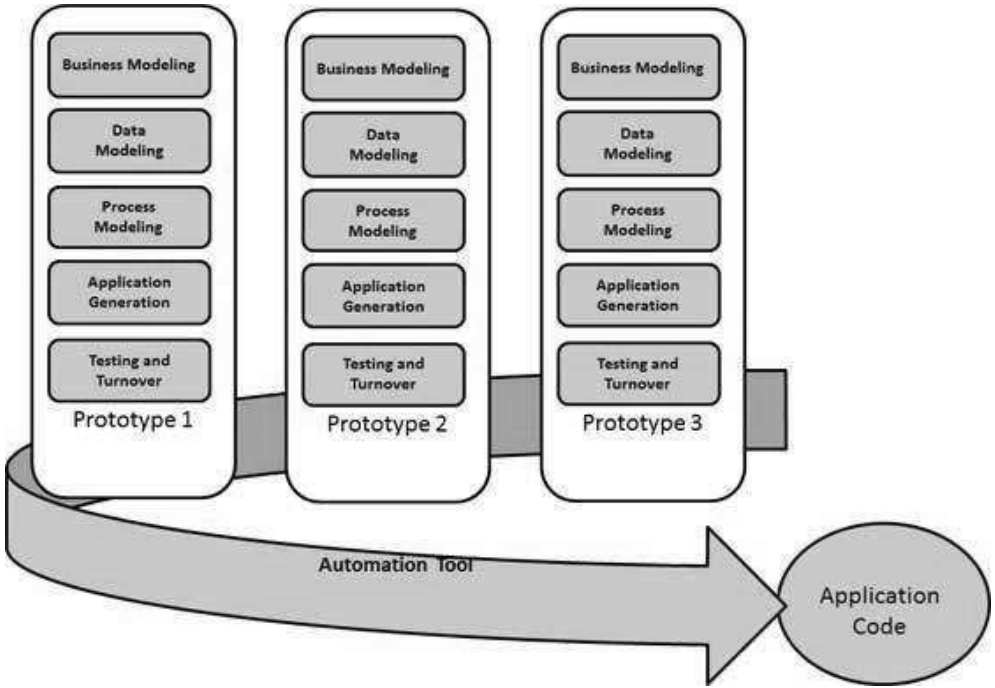


RAD (Rapid Application Development) Model

Following are the various phases of the RAD Model −

- **Business Modelling-** A complete business analysis is performed to find the vital information for business.
- **Data Modelling-** The relation between data objects are established and defined in detail in relevance to the business model.
- **Process Modelling-** The process model for any changes or enhancements to the data object sets is defined in this phase. Process descriptions for adding, deleting, retrieving or modifying a data object are given.
- **Application Generation-** The actual system is built and coding is done by using automation tools to convert process and data models into actual prototypes.
- **Testing and Turnover-** The overall testing time is reduced in the RAD model as the prototypes are independently tested during every iteration.

RAD (Rapid Application Development) Model



RAD (Rapid Application Development) Model

- **When to use RAD model:**
    - RAD should be used when there is a need to create a system that can be **modularized in 2-3 months of time.**
    - It should be used if there's **high availabilities of designers** for modelling and the **budget is high** enough to afford their cost along with the cost of automated code generating tools.
    - RAD SDLC model should be chosen **only if resources with high business knowledge** are available and there is a need to produce the system in a **short span of time** (2-3 months).

RAD (Rapid Application Development) Model

- **Advantages of the RAD model:**
  - Reduced development time.
  - Increases reusability of components
  - Quick initial reviews occur
  - Encourages customer feedback

- **Disadvantages of the RAD model:**
  - Depends on strong team and individual performances for identifying business requirements.
  - Requires highly skilled developers/designers.
  - High dependency on modeling skills
  - Inapplicable to cheaper projects as cost of modeling and automated code generation is very high.

VIT®
BHOPAL
www.vitbhopal.ac.in

RATIONAL UNIFIED PROCESS (RUP)

Prepared by: Dr. Rajneesh Kumar Patel

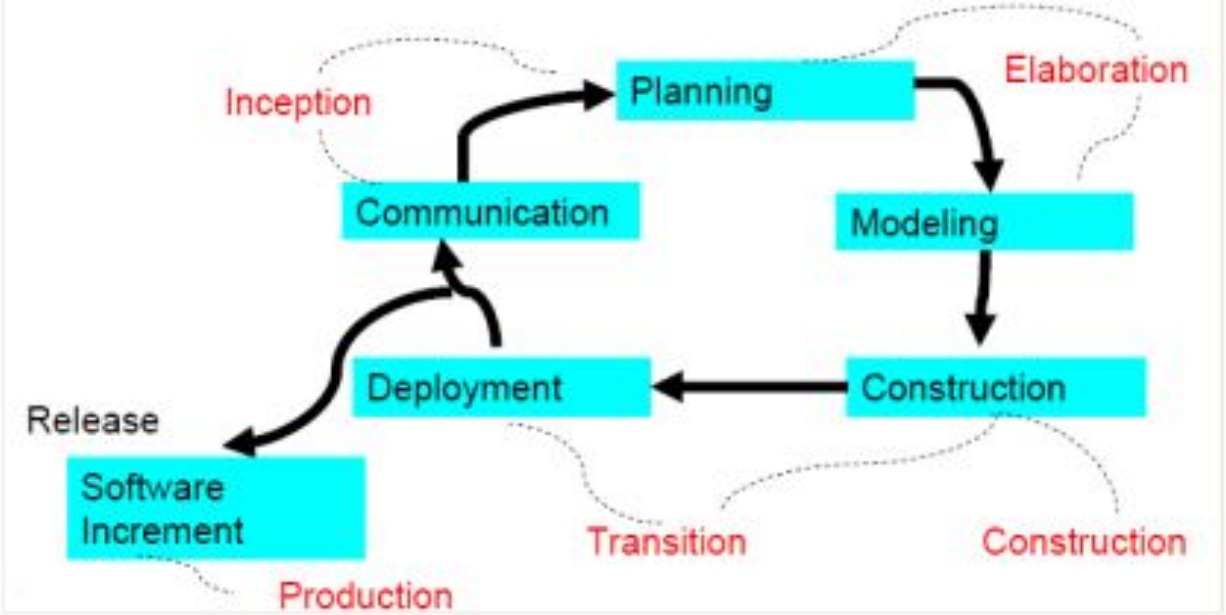
RATIONAL UNIFIED PROCESS (RUP)

- The **Unified Process** is an attempt to draw on the **best features and characteristics** of traditional software **process models**.
- It suggests a **process flow** that is **iterative and incremental**, providing the **evolutionary feel** that is essential in **modern software development**.
- During the early 1990s **James Rumbaugh**, **Grady Booch**, and **Ivar Jacobson** developed the Unified Process, a framework for object-oriented software engineering using **UML**.
- RUP is an **object-oriented approach** used to ensure effective project management and **high-quality software production.**

RATIONAL UNIFIED PROCESS (RUP)

- RUP divides the **development process** into **four distinct phases** that each **involves business modeling, analysis** and **design**, **implementation**, **testing**, and **deployment**.
- The four phases are:
    - **Inception** - The idea for the project is stated. The development team determines if the project is worth pursuing and what resources will be needed.
    - **Elaboration** - The project's architecture and required resources are further evaluated. Developers consider possible applications of the software and costs associated with the development.
    - **Construction** - The project is developed and completed. The software is designed, written, and tested.
    - **Transition** - The software is released to the public. Final adjustments or updates are made based on feedback from end users.

RATIONAL UNIFIED PROCESS (RUP)



RATIONAL UNIFIED PROCESS (RUP)

Advantages of RUP Software Development:

- This is a complete methodology in itself with an emphasis on accurate documentation
- It is proactively able to resolve the project risks associated with the client's evolving requirements requiring careful change request management
- The development time required is less due to reuse of components.
- There is online training and tutorial available for this process.

RATIONAL UNIFIED PROCESS (RUP)

- **Disadvantages of RUP Software Development:**
  - The **team members** need to be **expert** in their field to develop a software under this methodology.
  - The development process is too **complex and disorganized**.
  - On **cutting edge projects** which utilize new technology, the **reuse of components will not be possible.** Hence the time saving one could have made will be impossible to fulfill.
  - Integration throughout the process of software development, in theory sounds a good thing. But on particularly big projects with multiple development streams it will only add to the confusion and cause more issues during the stages of testing.

VIT®
BHOPAL
www.vitbhopal.ac.in

AGILE DEVELOPMENT MODEL

Prepared by: Dr. Rajneesh Kumar Patel

AGILE DEVELOPMENT MODEL

- The Word **“Agile”** means **think quickly and clearly.**
- Agile development model is also a type of **Incremental model.**
- Software is developed in incremental, **rapid cycles**.
- This results in small incremental releases with **each release building on previous functionality.**
- Each release is thoroughly tested to ensure software quality is maintained.
- Agile Model divides tasks into time boxes to provide specific functionality for the release.
- Each build is incremental in terms of functionality, with the final build containing all the attributes.

AGILE DEVELOPMENT MODEL

- The division of the entire project into small parts helps minimize the project risk and the overall project delivery time.

Important Announcement of Agile Model:

- Individuals and interactions are given priority over processes and tools.
- Focuses on working software rather than comprehensive documentation.
- Agile Model in software engineering aims to deliver complete customer satisfaction by rapidly delivering valuable software.
- Daily cooperation between business people and developers.
- It allows early and frequent delivery.
- **A strong emphasis is placed on face-to-face communication.**
- An improvement review is conducted regularly by the team.

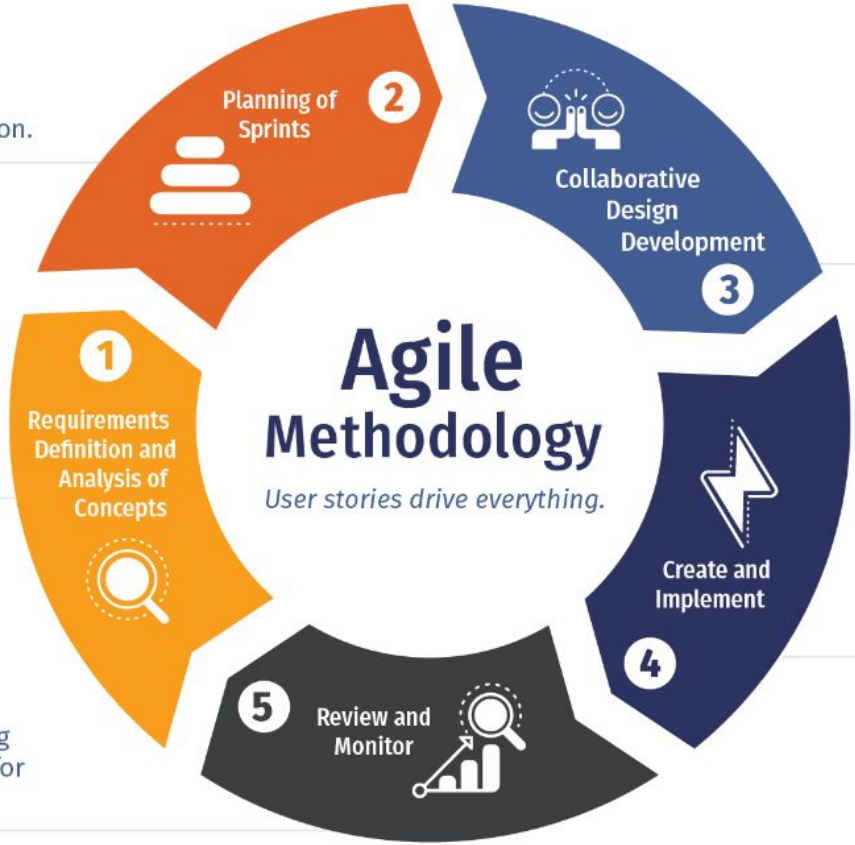
Phases of Agile Model:

Following are the phases in the Agile model are as follows:

1. Requirements gathering
2. Design the requirements
3. Construction/ iteration
4. Testing/ Quality assurance
5. Deployment
6. Feedback

Agile Methodology

User stories drive everything.



Analysis of concepts and requirements definitions; Determine current state and your expectations.

Arrange teams and tools needed to optimize production.

From the beginning of the process, the end users' involvement and feedback is critical.

Frequent development delivery through sprints. Feedback on testing & appropriate changes are imperative.

Ensure that you are reviewing and monitoring key metrics for success.

AGILE DEVELOPMENT MODEL

When to use the Agile Model?

- When frequent changes are required.
- When a highly qualified and experienced team is available.
- When a customer is ready to have a meeting with a software team all the time.
- When project size is small.

AGILE DEVELOPMENT MODEL

Advantages of the Agile Model

Here are some common pros/benefits of the Agile Model:

- Frequent Delivery
- Face-to-Face Communication with clients.
- Efficient design and fulfils the business requirement.
- Anytime changes are acceptable.
- It reduces total development time.

AGILE DEVELOPMENT MODEL

Disadvantages of Agile Model

Here are some common cons/drawbacks of the Agile Model:

- Documentation and design are not given much attention.
- Not a suitable method for handling complex dependencies.
- In some corporations, self-organization and intensive collaboration may not be compatible with their corporate culture.

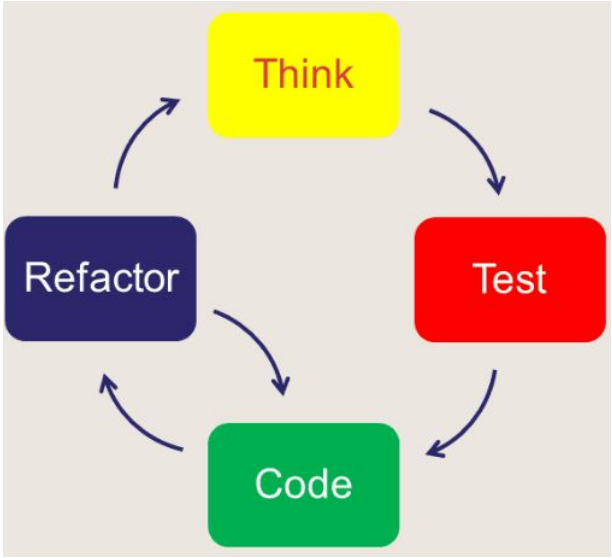
Agile Engineering Practices

- **Test Driven Development (TDD)**
- **Pair Programming**
- **Ensemble Programming**
- **Collective Code Ownership**
- **Continuous Integration and Deployment**

Agile Engineering Practices

Test Driven Development (TDD):

- TDD is a development practice where we start with a very small automated test about functionality we are about to build.
- The test fails because the code we are testing against does not yet exist. We then write enough code to make the test pass.
- Then we go through a very important step called refactoring. In refactoring, we clean up our code, updating and improving the internal implementation while all along ensuring that the test still passes.
- Once we are happy with the implementation, we move on and add another small test and we repeat.



- **Pair Programming:** Pair programming is a practice where 2 developers work together on the same problem at the same time. We have a driver that is writing the code and thinking tactically about the problem and a navigator that is observing and providing feedback while looking at the big picture.

- **Ensemble Programming:** Ensemble or mob programming takes pairing and turns the dial all the way up to 11. We are working on the problem together as a team. Instead of just sharing knowledge between 2 team members like in pairing, we are spreading that knowledge across the team.

- **Collective Code Ownership:** Collective Code Ownership means everybody on the team owns the code. We are no longer dependent on 1 person. We don't have to locally optimize our work based on who is available and what skills do they have. The entire team owns the code.

- **Continuous Integration and Deployment:** CI/CD is a practice that allows us to have the latest working version of our product up and running. It reinforces the approach of building applications iteratively and incrementally. Whenever someone makes a change, that change is integrated with what was already there and gets deployed.

Agile Testing Methods:

- **Scrum**
- **Crystal**
- **Dynamic Software Development Method(DSDM)**
- **Feature Driven Development(FDD)**
- **Lean Software Development**
- **eXtreme Programming(XP)**

Agile Testing Methods:

Scrum: is an agile development process focused primarily on ways to manage tasks in team-based development conditions.

There are three roles in it, and their responsibilities are:

- **Scrum Master:** The scrum can set up the master team, arrange the meeting and remove obstacles for the process
- **Product owner:** The product owner makes the product backlog, prioritizes the delay and is responsible for the distribution of functionality on each repetition.
- **Scrum Team:** The team manages its work and organizes the work to complete the sprint or cycle.

Agile Testing Methods:

Crystal:

There are three concepts of this method-

1. Chartering: Multi activities are involved in this phase such as making a development team, performing feasibility analysis, developing plans, etc.
2. Cyclic delivery: under this, two more cycles consist, these are:
    1. Team updates the release plan.
    2. Integrated product delivers to the users.
3. Wrap up: According to the user environment, this phase performs deployment, post-deployment.

Agile Testing Methods:

Dynamic Software Development Method(DSDM):

- DSDM is a rapid application development strategy for software development and gives an agile project distribution structure. The essential features of DSDM are that users must be actively connected, and teams have been given the right to make decisions.

- The techniques used in DSDM are:

1. Time Boxing
2. MoSCoW Rules
3. Prototyping

Agile Testing Methods:

- Feature Driven Development(FDD):

This method focuses on "Designing and Building" features. In contrast to other smart methods, FDD describes the small steps of the work that should be obtained separately per function.

- Lean Software Development:

Lean software development methodology follows the principle "just in time production." The lean method indicates the increasing speed of software development and reducing costs. Lean development can be summarized in seven phases.

- eXtreme Programming(XP)

This type of methodology is used when customers are constantly changing demands or requirements, or when they are not sure about the system's performance.